نمونه سوالات ساختمان داده ها

آرش همتی

پیچیدگی زمانی

بخش اول

پیچیدگی زمانی قطعه کدهای زیر را محاسبه کنید و Theta آن را پیدا کنید.

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        //Do an Operation
    }
}
```

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n ; j+=2) {
        //Do an Operation
    }
}
```

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n - 1; j++) {
        //Do an Operation
    }
}
```

```
for (int i = 0; i < n; i+=2) {
    for (int j = 0; j < n ; j+=2) {
        //Do an Operation
    }
}
```

پیچیدگی زمانی قطعه کدهای زیر را محاسبه کنید و Theta آن را پیدا کنید.

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n ; j *= 2) {
        //Do an Operation
    }
}
```

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n ; j++) {
        //Do an Operation
    }
    for (int j = 0; j < n; j++) {
        //Do and Operation
    }
}
```

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i ; j ++) {
        //Do an Operation
    }
}
```

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i ; j+=2) {
        //Do an Operation
    }
}
```

پیچیدگی زمانی قطعه کدهای زیر را محاسبه کنید و Theta آن را پیدا کنید.

```java
public static void operate(int n) {
    if(n > 1) {
        operate(n-1);
        System.out.println("Some Operation");
    }
}
```

```java
public static void operate(int n) {
    if(n > 1) {
        operate(n-2);
        System.out.println("Some Operation");
    }
}
```

```java
public static int operate(int n) {
    if(n == 1)
        return 1;
    else
        return 2 + operate(n/2);

}
```

```java
public static void operate(int n) {
    if(n == 1)
        System.out.println("Do operation");
    else {
        System.out.println("Do Operation");
        operate(n - 1) ;
        operate(n - 1);
    }
}
```

معادلات بازگشتی زیر را حل کنید. در همه معادلات زیر فرض کنید $t(0) = 0$ می باشد.

$t(n) = t(n-1) + 2$	$t(n) = t(n-2) + 3$
$t(n) = t(n-1) + 2n$	$t(n) = t(n-2) + n$
$t(n) = t(n-1) + \frac{1}{n}$	$t(n) = t\left(\frac{n}{2}\right) + 1$
$t(n) = 2t(n-1) + \frac{2^n}{n}$	$t(n) = t\left(\frac{n}{2}\right) + \frac{n}{2}$

متدهای بازگشتی

بخش دوم

متد بازگشتی بنویسید که دو عدد a, b **را از ورودی بگیرد و مقدار** a^b **را محاسبه نماید و بازگرداند.**

نام متد	power
ورودی	int,int
خروجی	int

$$\text{power}(a,b) = \begin{cases} 1 & if\ b = 0 \\ \text{a} * \text{power}(a, b-1) & if\ b > 0 \end{cases}$$

متد بازگشتی بنویسید که دو عدد a, b **را از ورودی بگیرد و مقدار بزرگترین مقسوم علیه مشترک آنها را محاسبه نماید و بازگرداند.**

نام متد	gcd
ورودی	int,int
خروجی	int

$$\gcd(a,b) = \begin{cases} a & if\ b = 0 \\ \gcd(b\ , a\%b) & if\ b > 0 \end{cases}$$

متد بازگشتی بنویسید که عدد n را از ورودی بگیرد و مجموع ارقام آن را محاسبه نماید و بازگرداند.

نام متد	sumDigits
ورودی	int
خروجی	int

$$\text{sumDigits(n)} = \begin{cases} 1 & if\ \frac{n}{10} = 0 \\ 1 + \text{sumDi}gits(n/10) & else \end{cases}$$

متد بازگشتی بنویسید که دو عدد n, r را از ورودی بگیرد و مقدار ترکیب r از n را محاسبه کرده و بازگرداند.

نام متد	combine
ورودی	int,int
خروجی	int

$$\binom{n}{r} = \begin{cases} 1 & if\ r = 0\ or\ r = n \\ \binom{n-1}{r} + \binom{n-1}{r-1} & otherwise \end{cases}$$

متد بازگشتی بنویسید که عدد n را از ورودی بگیرد و مقدار فاکتوریل n را محاسبه کرده و بازگرداند.

نام متد	fact
ورودی	int
خروجی	int

$$n! = \begin{cases} 1 & if\ n = 0 \\ n * (n-1)! & if\ n > 0 \end{cases}$$

متد بازگشتی بنویسید که عدد n را از ورودی بگیرد و مقدار n ام سری فیبوناتچی را بازگرداند.

$$fibonacci\ series = 1,1,2,3,5,8,13,21,34, \ldots$$

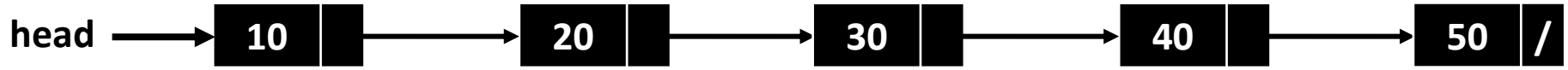
نام متد	fib
ورودی	int
خروجی	int

$$fib(n) = \begin{cases} 1 & if\ n = 1\ or\ n = 2 \\ fib(n-1) + fib(n-2) & if\ n > 2 \end{cases}$$

لیست پیوندی

بخش سوم

برای پیاده سازی متدهای این بخش فرض کنید که کلاس لیست پیوندی به صورت زیر تعریف شده است.




```java
public class Node {
    private int data;
    private Node next;

    public Node(int data) {
        this.data = data;
    }

    //getters and setters
}
```

```java
public class LinkedList {
    private Node head;
    private int size = 0;
}
```

در کلاس لیست پیوندی متدی بنویسید که شماره اندیس یک گره را بگیرد و مقدار آن را بازگرداند (اندیس گره ها از صفر شروع می شود).

نام متد	getValueAt
ورودی	int
خروجی	int

در کلاس لیست پیوندی متدی بنویسید که اعداد n, val **را از ورودی بگیرد و گره با مقدار** val **را در اندیس شماره** n **درج کند. (اندیس گره ها از صفر شروع می شود).**

نام متد	addNodeAt
ورودی	int,int
خروجی	void

در کلاس لیست پیوندی متدی بنویسید که شماره اندیس n را از ورودی بگیرد و آن گره را حذف کند. (اندیس گره ها از صفر شروع می شود).

deleteNodeAt	نام متد
int	ورودی
void	خروجی

در کلاس لیست پیوندی متدی بنویسید که مقدار val **را از ورودی بگیرد و آن را در لیست پیدا کرده و اندیس اولین گره حاوی عدد** val **را بازگرداند. (اندیس گره ها از صفر شروع می شود).**

نام متد	find
ورودی	int
خروجی	int

در کلاس لیست پیوندی متدی بنویسید که کل لیست پیوندی را خالی کند.

نام متد	clear
ورودی	-
خروجی	void

در کلاس لیست پیوندی متدی بنویسید که یک عدد به نام val **را از ورودی بگیرد و اگر آن عدد در لیست وجود داشت مقدار** true **و در غیر اینصورت مقدار** false **را بازگرداند.**

نام متد	contains
ورودی	int
خروجی	boolean

در کلاس لیست پیوندی متدی بنویسید که یک عدد به نام val **را از ورودی بگیرد و آن عدد را در یک گره جدید در ابتدای لیست اضافه کند.**

addFirst	نام متد
int	ورودی
void	خروجی

در کلاس لیست پیوندی متدی بنویسید که یک عدد به نام val **را از ورودی بگیرد و آن عدد را در یک گره جدید در انتهای لیست اضافه کند.**

نام متد	addLast
ورودی	int
خروجی	void

در کلاس لیست پیوندی متدی بنویسید که یک عدد به نام n را از ورودی بگیرد و n گره انتهایی لیست را حذف کند (راهنمایی: از ویژگی size که در لیست پیوندی تعریف کرده ایم کمک بگیرید).

نام متد	cutLast
ورودی	int
خروجی	void

پشته (Stack)

بخش چهارم

برای پیاده سازی متدهای این بخش فرض کنید که کلاس استک با آرایه و به صورت زیر تعریف شده است.

```java
public class Stack {
    private int[] arr;
    private int top;

    public Stack(int size) {
        this.arr = new int[size];
        this.top = -1;
    }
}
```



در کلاس استک که با آرایه پیاده سازی شده است متد push را پیاده سازی نمایید

نام متد	push
ورودی	int
خروجی	void

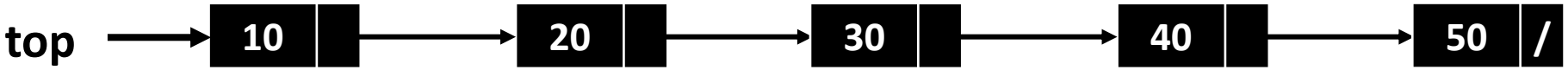
در کلاس استک که با آرایه پیاده سازی شده است متد pop را پیاده سازی نمایید

نام متد	pop
ورودی	-
خروجی	int

در کلاس استک که با آرایه پیاده سازی شده است متد peek را پیاده سازی نمایید

نام متد	peek
ورودی	-
خروجی	int

برای پیاده سازی متدهای این بخش فرض کنید که کلاس استک با لیست پیوندی و به صورت زیر تعریف شده است.




```
public class Stack {
    private Node top;
}
```

در کلاس استک که با لیست پیوندی پیاده سازی شده است متد push را پیاده سازی نمایید

نام متد	push
ورودی	int
خروجی	void

در کلاس استک که با لیست پیوندی پیاده سازی شده است متد pop را پیاده سازی نمایید

نام متد	pop
ورودی	-
خروجی	int

در کلاس استک که با لیست پیوندی پیاده سازی شده است متد peek را پیاده سازی نمایید

نام متد	peek
ورودی	-
خروجی	int

فرمهای نمایشی و ارزیابی

بخش پنجم

فرم های نمایشی میانوندی زیر را به فرم پیشوندی و پسوندی تبدیل نمایید

Infix
a + b * c / d + e - f
(a + b) * c / d + e - f
(a + b) * c / (d + e) - f
a ^ b * c * d * f - g * h
a * b ^ c * d + (f - g) * h ^ i

فرم های نمایشی پسوندی زیر را به فرم میانوندی تبدیل نمایید

Postfix
ab+c/fg+-
ab*c-fgh^*-
abcd*f-/+
ab*cd*/ef^g*hi+/+

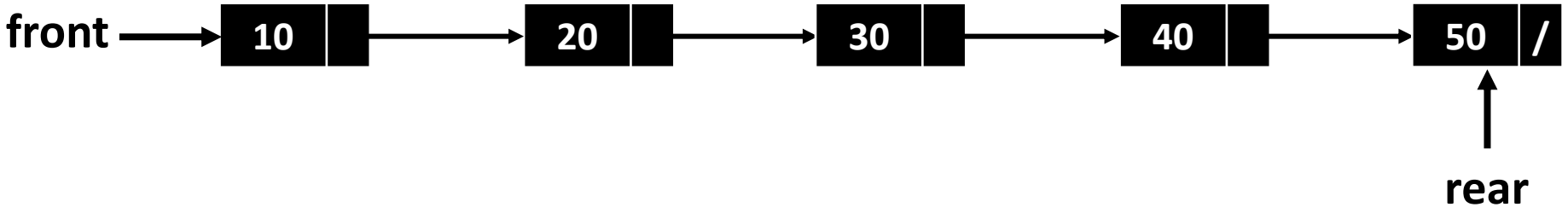
با استفاده از الگوریتم تبدیل میانوندی به پسوندی و نیز الگوریتم ارزیابی عبارات ریاضی نشان دهید که نتیجه عبارت ریاضی زیر چگونه محاسبه می شود.

$$\frac{20 + 30}{5 * 2} + 4^2$$

صف

بخش ششم

برای پیاده سازی متدهای این بخش فرض کنید که کلاس صف با لیست پیوندی و به صورت زیر تعریف شده است.




```
public class Queue {
    private Node front;
    private Node rear;
}
```

در کلاس صف که با لیست پیوندی پیاده سازی شده است متد enqueue را پیاده سازی نمایید

نام متد	enqueue
ورودی	int
خروجی	void

در کلاس صف که با لیست پیوندی پیاده سازی شده است متد dequeue را پیاده سازی نمایید

نام متد	dequeue
ورودی	–
خروجی	int

در کلاس استک که با لیست پیوندی پیاده سازی شده است متد peek را پیاده سازی نمایید

نام متد	peek
ورودی	-
خروجی	int

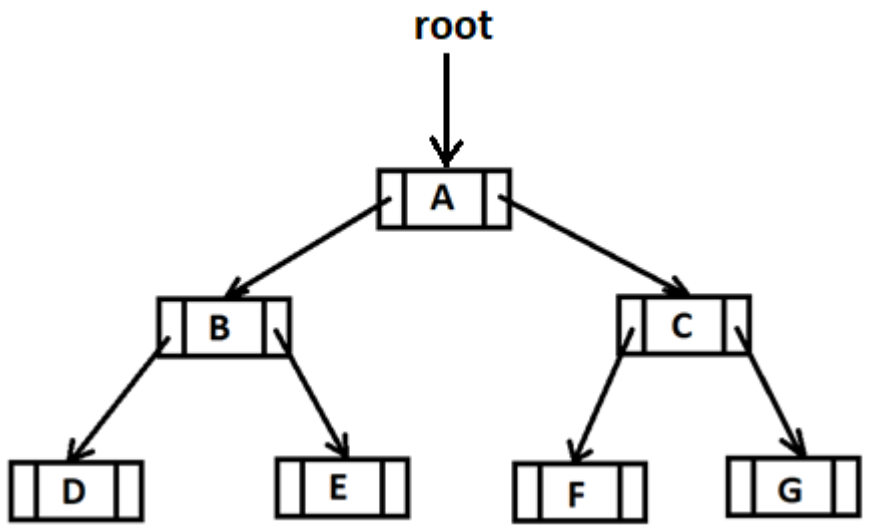
درخت های دودویی

بخش هفتم

برای پیاده سازی متدهای این بخش فرض کنید که کلاس درخت دودویی با لیست پیوندی دوطرفه و به صورت زیر تعریف شده است.

```
public class Node {
    private int data;
    private Node left;
    private Node right;

    public Node(int data) {
        this.data = data;
    }
}
public class Tree {
    private Node root;
}
```



در کلاس درخت دودویی متد پیمایش پیش ترتیب را پیاده سازی نمایید

نام متد	preOrder
ورودی	Node
خروجی	-

در کلاس درخت دودویی متد پیمایش میان ترتیب را پیاده سازی نمایید

نام متد	inOrder
ورودی	Node
خروجی	-

در کلاس درخت دودویی متد پیمایش پس ترتیب را پیاده سازی نمایید

postOrder	نام متد
Node	ورودی
-	خروجی

در کلاس درخت دودویی متدی بنویسید که تعداد گره های درخت را بازگرداند (راهنمایی: بازگشتی پیاده سازی کنید)

countNodes	نام متد
Node	ورودی
int	خروجی

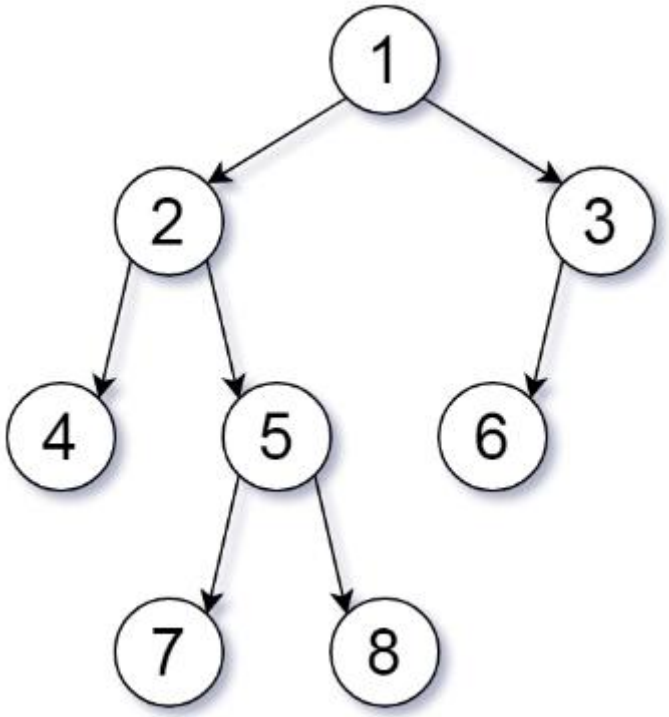
در کلاس درخت دودویی متدی بنویسید که تعداد برگ های درخت را بازگرداند (راهنمایی: بازگشتی پیاده سازی کنید)

countLeaves	نام متد
Node	ورودی
int	خروجی

در کلاس درخت دودویی متدی بنویسید که ارتفاع درخت را بازگرداند (راهنمایی: بازگشتی پیاده سازی کنید)

نام متد	findHeight
ورودی	Node
خروجی	int

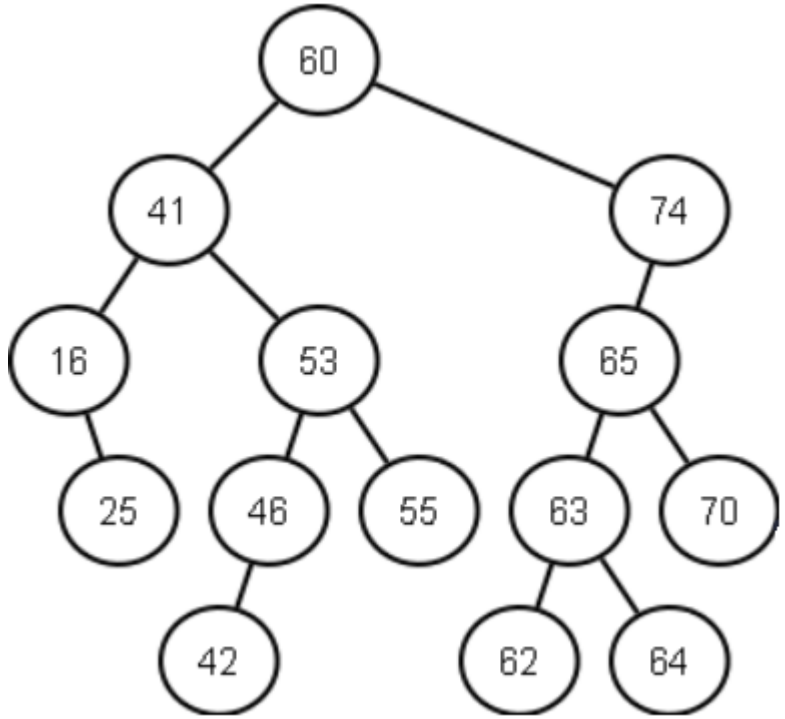
درخت زیر را به صورت پیش ترتیب، میان ترتیب و پس ترتیب پیمایش کنید



پیمایش های زیر مربوط به یک درخت است، آن درخت را ترسیم نمایید.

In Order	**E C B D F A J H K G L I M**
Post Order	**E C F D B J K H L M I G A**

درخت جستجوی دودویی زیر پس از حذف گره های ۶۲ سپس ۷۴ و در انتها ۴۱ به چه شکلی در خواهد آمد و پس از حذف گره ها ارتفاع درخت چقدر خواهد شد؟



گره های جدول زیر را به ترتیب از چپ به راست در یک درخت جستجوی دودویی وارد نمایید و درخت را ترسیم کنید.

10 – 11 – 15 – 7 – 5 – 8 – 4 – 13 - 6 – 1 – 14 – 9 – 3 – 12 - 2

گره های زیر را در یک درخت AVL وارد نمایید (چرخش های لازم را انجام دهید).

10-13-15-18-11-14-16

گره های زیر را در یک درخت Max Heap وارد نمایید (مرحله به مرحله بیان کنید که چه درختی حاصل می شود).

10-13-15-18-11-14

اعداد زیر را (از چپ به راست) با استفاده از الگوریتم Heap Sort مرتب سازی کنید

15-14-18-16-12-17-11

در جدول زیر تعدادی کاراکتر و فرکانس آنها داده شده است. با استفاده از الگوریتم هافمن یک سیستم کدگذاری فشرده شده برای آن ارائه نمایید (مرحله به مرحله توضیح دهید) و بیان کنید که چند بیت صرفه جویی صورت گرفت

A	B	C	D	E	F	G
5	4	4	3	3	1	2

با آرزوی موفقیت برای شما

پایان